Rings: An Algebraic Structured Peer-to-Peer Network for Decentralized Identity and Resource Ownership

Ryan J. Kung

Abstract

Rings is an implemented algebraic structured peer-to-peer network for decentralized identity, resource ownership, transport, storage, and application protocols. The project starts from a concrete failure mode in contemporary Internet systems: application servers, DNS names, relay services, and platform accounts often become the authority that identities and data must trust. Rings instead treats identities and resources as algebraic objects over a structured CorrectChord overlay, with browser and native runtimes connected through WebRTC/WebAssembly adapters and cryptographic protocols kept on carriers separate from the routing identifier space. The core object is

$$\mathcal{R}\rangle\backslash\}\int = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ \Omega_N(X,H) &= (X, \mathrm{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

$$\{D, V, M, H, Q, W\} \subset \mathrm{Obj}(\mathbf{Set}/R), \qquad \mathsf{CryptoCarrier} \cap R = \varnothing.$$

I. Introduction

Centralized application platforms simplify deployment, but they also concentrate identity, naming, storage, and traffic control at service operators. Cloud and platform centralization therefore becomes a privacy, availability, and autonomy risk for users who need direct communication and durable control over their data [1]–[3]. The Rings project is motivated by this observation: a sovereign network should let a participant own an identity, establish a session, route to another identity or resource, and run application protocols without turning a single application server into the root of authority.

Rings is implemented as a network, but this paper is organized around the algebraic structure that makes the implementation coherent. It states the ideal carrier, ownership, transition, and safety laws that the implementation realizes and continues to refine. The older intuition was that Decentralized Identifiers (DIDs) directly form a finite ring suitable for all routing and cryptographic operations. The refined model is more precise: Chord identifiers form an additive circular carrier for placement, interval order, replica selection, and finger targets; threshold schemes, signatures, encryption, and zero-knowledge protocols use their own fields or groups.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created February 8, 2023; last updated July 4, 2026.

The resulting design has three practical consequences. First, DIDs and virtual DIDs (VIDs) share the same 160-bit placement surface, so identities, mailboxes, service descriptors, chunks, subrings, and application rendezvous points can be routed by one owner function. Second, Rings builds on Chord and CorrectChord rather than replacing the overlay literature: successor lists, predecessor relations, join, rectify, and stabilize are the maintenance substrate on which Rings semantics are layered [4], [5]. Third, state transitions return typed effects instead of directly performing network IO, which allows the overlay to be reasoned about as a pure state machine and then interpreted by browser or native runtimes.

II. Problem Statement and Design Goals

Rings addresses a narrow but recurring gap between existing decentralized systems. Kademlia-derived systems provide scalable lookup, but the XOR metric does not by itself give Rings the circular ownership law needed for successor lists, replicated ownership, and interval-based placement. Relay networks and mixnets improve metadata resistance, but they are not structured P2P ownership systems. Blockchains provide global ordering, but embedding consensus in every message or storage operation would turn Rings into a ledger rather than an algebraic structured peer-to-peer substrate.

The problem is therefore to specify a network object

$$\mathcal{R}\rangle\backslash\}\int$$

that simultaneously supports:

1) self-certifying identity and delegated sessions;
2) structured lookup and resource ownership;
3) browser-native and native transport surfaces;
4) application protocols with typed side effects;
5) cryptographic protocols over correct carriers;
6) bounded algebraic and operational obligations for routing and state repair.

This paper makes the following contributions. It presents Rings as an algebraic structured peer-to-peer network, not merely as an overlay lookup routine. It models the Rings identifier space as the additive cyclic group $R = \mathbb{Z}/2^{160}\mathbb{Z}$, not as a general finite field. It lifts resource placement into $\mathbf{Set}/R$, so each protocol object carries an explicit map into the overlay. It uses CorrectChord as the maintenance law before introducing Rings-specific replication, storage, mailbox, relay, and ranking semantics. It gives a writer effect monad for DHT transitions and a TLA-style state relation for model checking and trace replay. These algebraic and operational artifacts are the core of the paper; the surrounding text explains why those artifacts exist and how they relate to prior systems.

III. Algebraic System Model

Assumption 1 (System scope).

$$\begin{aligned} \mathsf{fault} &= \mathsf{fail\text{-}stop}, \\ \mathsf{auth}(m,\sigma,k) &= V(k,\sigma,m), \\ \mathsf{net} &= \mathsf{async}, \\ \mathsf{io} &= I_X : A_X^* \to \mathsf{IO}, \\ \mathsf{consensus} &\notin \mathsf{Overlay}, \\ \mathsf{order} &= \mathsf{SidecarWitness}, \\ \mathsf{crypto\_ops} &\cap \mathsf{route\_ops} = \varnothing. \end{aligned}$$

Definition 1 (Identifier carrier).

$$\begin{aligned} M &= 2^{160}, \\ R &= \mathbb{Z}/M\mathbb{Z}, \\ \delta_a(x) &= (x-a) \bmod M, \\ (a,b]_R &= \{x \in R : 0 < \delta_a(x) \le \delta_a(b)\}, \\ \mathsf{RouteOps}_R &= \{0,+,-,\delta_a,(\_,\_]_R\}, \\ \mathsf{Mult}_R &\notin \mathsf{RouteOps}_R. \end{aligned}$$

Definition 2 (Live topology).

$$\begin{aligned} N &\subset R, \quad N \neq \varnothing, \\ \mathrm{owner}_N(k) &= \arg\min_{u \in N} \delta_k(u), \\ \mathrm{pred}_N(n) &= \arg\min_{p \in N\setminus\{n\}} \delta_p(n), \\ \mathrm{Succ}_N^q(n) &= \mathrm{take}_q\big(\mathrm{sort}_{\delta_n}(N \setminus \{n\})\big), \\ \delta_a|_N &= \mathsf{injective}. \end{aligned}$$

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned} r &= |\mathrm{Succ}_N^r(n)|, \\ \mathsf{Steady}(N) &\Rightarrow |N| \ge r+1, \\ |N| < r+1 &\Rightarrow \mathsf{Init}(N), \\ \mathsf{Maintain} &= \{\mathsf{join},\mathsf{rectify}\} \\ &\quad \cup \{\mathsf{stabilize}\}_{CorrectChord}. \end{aligned}$$

This is the CorrectChord stable base of Zave [5].

Definition 3 (Replica set).

$$\mathrm{Rep}_N^r(k) = \{\mathrm{owner}_N(k)\} \cup \mathrm{set}\big(\mathrm{Succ}_N^{r-1}(\mathrm{owner}_N(k))\big).$$

Definition 4 (Rings overlay object).

$$\mathcal{R}_N = (R, N, \mathrm{owner}_N, \mathrm{pred}_N, \mathrm{Succ}_N^r, \mathrm{Rep}_N^r, F, E),$$

$$\begin{aligned} F &\subseteq N \times \mathbb{N} \times N, \\ E &= \textstyle\prod_{v \in VID} E_v, \\ \mathsf{SafetyRoot}(\mathcal{R}_N) &= (\mathrm{pred}_N, \mathrm{Succ}_N^r), \\ \mathsf{PerfWitness}(\mathcal{R}_N) &= F. \end{aligned}$$

Proposition 1 (Rotation equivariance).

$$t \in R, \quad N + t = \{n + t : n \in N\} \Rightarrow$$
$$\mathrm{owner}_{N+t}(k+t) = \mathrm{owner}_N(k) + t.$$

Proof.

$$\begin{aligned} \forall n \in N. \quad \delta_{k+t}(n+t) &= ((n+t)-(k+t)) \bmod M \\ &= \delta_k(n). \end{aligned}$$

□

Constraint 1 (Carrier separation).

$$\begin{aligned} \mathsf{PlacementOps} &= \{0,+,-,\delta_a,(a,b]_R\} \\ &\quad \cup \{\mathrm{owner}_N, \mathrm{Rep}_N^r\}, \\ \mathsf{CryptoOps} &= \{\times, ^{-1}, \cdot_{scalar}, \mathsf{interp}\} \\ &\quad \cup \{\mathsf{pair},\mathsf{subgroup}\}, \\ \mathsf{PlacementOps} &\cap \mathsf{CryptoOps} = \varnothing, \\ \mathsf{CryptoCarrier} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}. \end{aligned}$$

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID placement	$H_\nu : X_\nu \to R$ then owner map
Storage merge	join semilattice or explicit finalizer
E2E encryption	finite field or curve group
Sequencing	partial order over witness domains
Effects	writer monad over action lists

Table I
Carrier boundaries in the Rings model

IV. Architecture As Interfaces

Definition 5 (Layer tuple).

$$\mathcal{A} = (I, T, O, X, P, L)$$

I	$(DID, \Pi, \mathrm{Session})$
T	$(C, A_T, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$
O	$\mathcal{R}_N$
X	$(S_X, A_X, \mathsf{cap}, I_X)$
P	$\{\mathcal{P}_\nu\}_{\nu \in Namespace}$
L	$\mathsf{App} \circ P$

Definition 6 (Identity proof).

$$\Pi = (K, \Sigma, V, D, \mathsf{enc}, \mathsf{ttl})$$

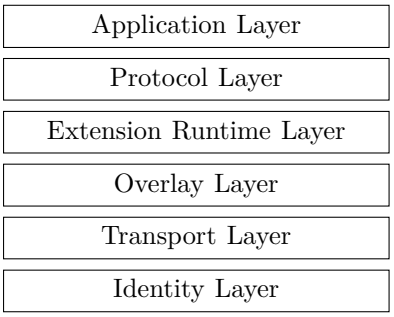


Figure 1. Layers of Rings Network

$$\begin{aligned} V &: K \times \Sigma \times B^* \to \{\top, \bot\}, \\ D &: K \to R, \\ \mathsf{enc} &: \Sigma \to B^*, \\ \mathsf{ttl} &: \Sigma \to Time. \end{aligned}$$

Invariant 1 (Session authority).

$$d \in X_D, \qquad s \in K.$$

$$\text{Session}(d, s, e) \Rightarrow V(k_d, \sigma_{d \to s}, d \| s \| e) = \top \wedge now < e.$$

$$\mathsf{HopCheck}(d, s, e) \equiv \text{Session}(d, s, e).$$

Definition 7 (Transport interface).

$$T = (C, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$$

$$A_T = \{\mathsf{Offer}, \mathsf{Answer}, \mathsf{Ice}, \mathsf{Open}, \mathsf{Send}, \mathsf{Recv}, \mathsf{Close}\}.$$

$$I_T : A_T^* \to \mathsf{IO}_{WebRTC} \cup \mathsf{IO}_{Native}.$$

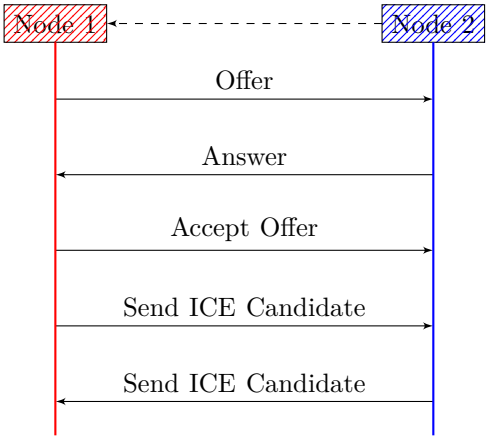


Figure 2. Standard SDP/ICE exchange

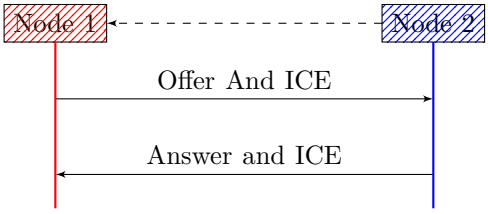


Figure 3. Rings single round-trip SDP/ICE exchange

Definition 8 (Runtime interpreter).

$$\mathcal{X} = (S_X, A_X, \mathsf{cap}, I_X)$$

$$\begin{aligned} \mathsf{cap} &\subseteq A_X, \\ I_X &: A_X^* \to \mathsf{IO}, \\ I_X(\epsilon) &= \mathsf{Noop}, \\ I_X(\alpha \cdot \beta) &= I_X(\alpha); I_X(\beta). \end{aligned}$$

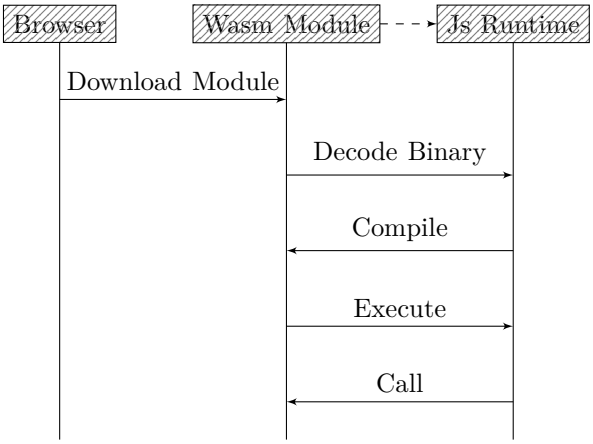


Figure 4. Wasm module execution boundary

Obligation 1 (Runtime confinement).

$$\begin{aligned} \tau &: S \times I \to \mathcal{E} + (S \times O \times A_X^*), \\ \mathsf{emit}(\tau(s, i)) &\subseteq \mathsf{cap}, \\ a \notin \mathsf{cap} &\Rightarrow I_X(a) = \mathsf{Reject}(a). \end{aligned}$$

V. Category and Effects

Definition 9 (Namespace object).

$$\begin{aligned} \nu &\in \{D, V, M, H, U, Q, W\}, \\ H_\nu &: X_\nu \to R, \\ (X_\nu, H_\nu) &\in \mathbf{Set}/R. \end{aligned}$$

Definition 10 (Placement functor).

$$\begin{aligned} \Omega_N &: \mathbf{Set}/R \to \mathbf{Set}/N, \\ (X, H) &\mapsto (X, \text{owner}_N \circ H). \end{aligned}$$

$$P_\nu^N = \text{owner}_N \circ H_\nu : X_\nu \to N.$$

Proposition 2 (Placement functoriality).

$$f : X_\nu \to X_\mu, \qquad H_\mu \circ f = H_\nu.$$

$$P_\mu^N \circ f = P_\nu^N.$$

Proof.

$$\text{owner}_N \circ H_\mu \circ f = \text{owner}_N \circ H_\nu.$$

□

Definition 11 (Writer effect monad).

$$\begin{aligned} A^* &= \mathsf{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg\!= f &= \mathsf{let}\ f(x) = (y, \beta)\ \mathsf{in}\ (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 3 (Effect associativity).

$$(h^\star \circ g^\star) \circ f^\star = h^\star \circ (g^\star \circ f^\star).$$

Proof.

$$((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma)).$$

□

Definition 12 (Typed failure stack).

$$\mathsf{T}_A^{\mathcal{E}}(X) = \mathcal{E} + (X \times A^*).$$

$$\begin{aligned} \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \mathsf{RecoverableReport} &\in A. \end{aligned}$$

Definition 13 (Protocol object).

$$\mathcal{P}_\nu = (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu)$$

$$\tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu).$$

Definition 14 (Protocol morphism).

$$\Sigma : \mathcal{P}_\nu \to \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*)$$

$$\begin{aligned} \sigma_A^* &: A_\nu^* \to A_\mu^*, \\ \sigma_A^*(\alpha \cdot \beta) &= \sigma_A^*(\alpha) \cdot \sigma_A^*(\beta), \\ H_\mu \circ \sigma_X &= H_\nu, \\ \mathsf{T}_{\sigma_A}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu &= \tau_\mu \circ (\sigma_S \times \sigma_I). \end{aligned}$$

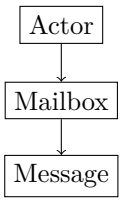


Figure 5. Actor Model

VI. Chord Overlay

$$\begin{aligned} \mathsf{Base} &= \mathsf{Chord}, \\ \mathsf{Maintenance} &= \mathsf{CorrectChord}, \\ \mathsf{Object} &= \mathcal{R}_N, \\ \mathsf{Witnesses} &= \{\text{Algorithms } 1, 2, 3\}. \end{aligned}$$

This section fixes Chord as the base protocol and CorrectChord as the maintenance law [4], [5].

Definition 15 (Local node state).

$$S_n = (p_n, L_n, F_n, E_n)$$

$$\begin{aligned} p_n &\in N \cup \{\bot\}, \\ L_n &\in N^{\leq r}, \\ F_n &: \mathbb{N} \rightharpoonup N, \\ E_n &\subseteq E, \\ p_n = \mathrm{pred}_N(n), &\qquad L_n = \mathrm{Succ}_N^r(n). \end{aligned}$$

Definition 16 (Finger target).

$$\begin{aligned} \mathrm{target}_i(n) &= n + 2^i, \\ \mathrm{finger}_i(n) &= \mathrm{owner}_N(\mathrm{target}_i(n)), \\ F_n(i) &\in \{\bot, \mathrm{finger}_i(n)\}. \end{aligned}$$

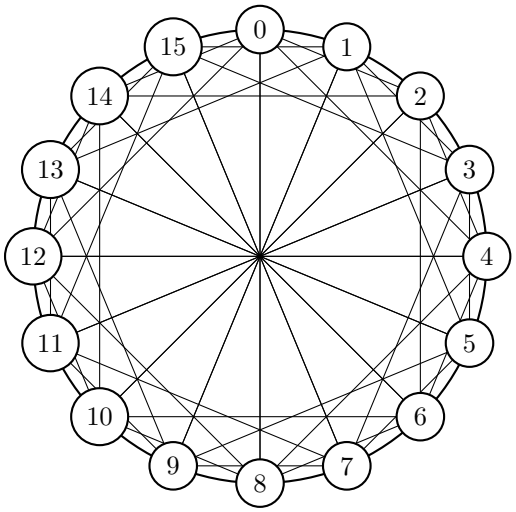


Figure 6. Chord algorithm, lookup protocol

Algorithm 1 DHT Join Transitions

```
1: function BeginJoin(n, known)
2:     pred[n] ← none
3:     a ← FindSuccessor(n.did)
4:     return Remote(known, a)
5: function InstallSuccessor(n, s)
6:     succ[n] ← take(r, [s])
7:     return Remote(s, QuerySuccList(s))
8: function InstallSuccList(n, s, list)
9:     succ[n] ← take(r, s :: list)
10:    if head(succ[n]) ≠ none then
11:        h ← head(succ[n])
12:        notify ← Notify(h, n.did)
13:        sync ← SyncStorage(h)
14:        return Multi(notify, sync)
15:    else
16:        return Noop
```

Invariant 2 (Ring fixpoint).

$$\mathrm{Inv}_{ring}(N, S) \equiv \forall n \in N.\ p_n = \mathrm{pred}_N(n) \wedge L_n = \mathrm{Succ}_N^r(n).$$

Invariant 3 (Lookup owner preservation).

$$\mathsf{Steady}(N) \wedge \mathrm{Inv}_{ring}(N, S) \Rightarrow \mathrm{Lookup}(S, k) \Downarrow \mathrm{owner}_N(k).$$

$$\Downarrow = \mathsf{Reach}_{A_D}(2, 3).$$

VII. DID and Resource Algebra

Definition 17 (DID namespace).

$$\mathcal{D} = (X_D, H_D, \Pi, \mathrm{Session}), \qquad H_D : X_D \to R.$$

$$\begin{aligned} D(k) &= H_D(k), \\ \mathrm{owner}_N(D(k)) &\in N, \\ \mathrm{Session} &\subseteq X_D \times K \times Time. \end{aligned}$$

Algorithm 2 DHT Lookup Algorithm

```
function Lookup(key, node)
    s ← head(succ[node])
    if s = none then
        return Repair(QuerySuccList(node))
    if key ∈ (node.did, s] then
        return Local(s)
    next ← closest_preceding_node(node, key)
    if next = none then
        return Repair(QuerySuccList(s))
    else
        a ← FindSuccessor(key)
        return Remote(next, a)
```

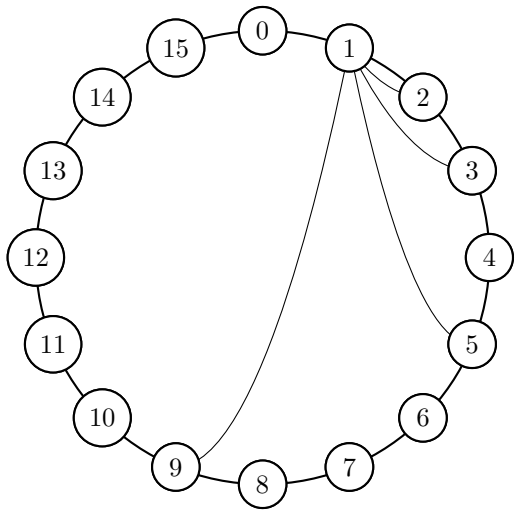


Figure 7. Chord algorithm, lookup protocol

Definition 18 (VID namespace).

$$\begin{aligned} \mathcal{V} &= (X_V, H_V, \mathrm{Auth}_V), \\ H_V &: X_V \to R, \\ \mathrm{PrivateKey}(H_V(x)) &= \bot, \\ \mathrm{Auth}_V(op, x, \sigma, caps) &\in \{\top, \bot\}. \end{aligned}$$

Definition 19 (Biased order).

$$A \leq_C B \iff \delta_C(A) \leq \delta_C(B).$$

Definition 20 (Replicated entry algebra).

$$\begin{aligned} \mathcal{E}_v^{join} &= (E_v, \sqsubseteq, \sqcup, \bot), \\ \mathcal{E}_v^{final} &= (E_v, \mathsf{conflict}_v, \mathsf{finalize}_v). \end{aligned}$$

$$\begin{aligned} x \sqcup y &= y \sqcup x, \\ (x \sqcup y) \sqcup z &= x \sqcup (y \sqcup z), \\ x \sqcup x &= x, \\ x \sqcup \bot &= x, \\ \mathsf{finalize}_v &: \mathsf{conflict}_v(E_v) \to E_v. \end{aligned}$$

$$\mathsf{case}_1 = \mathsf{CRDT}.$$

This is the CRDT join-semilattice case [6].

Algorithm 3 DHT Stabilization

```
procedure Stabilize(n, topo)
    actions ← []
    c ← topo.pred
    s ← head(succ[n])
    if c ≠ none and c ∈ (n.did, s) then
        succ[n] ← insert(succ[n], c)
        actions ← actions :: FindSuccessor(c)
    succ[n] ← take(r, merge(succ[n], topo.succ))
    if head(succ[n]) ≠ none then
        h ← head(succ[n])
        actions ← actions :: Notify(h, n.did)
    return Multi(actions)
```

Algorithm 4 Biased Circular Comparison

```
function BiasedCompare(A, B, C)
    a ← δ_C(A)
    b ← δ_C(B)
    if a < b then
        return A <_C B
    else if a > b then
        return B <_C A
    else
        return A = B
```

Invariant 4 (Placement).

$$\mathrm{Inv}_{place} \equiv \forall \nu, x.\ \mathrm{replicas}(H_\nu(x)) \subseteq \mathrm{Rep}_N^r(H_\nu(x)).$$

Definition 21 (Mailbox).

$$\begin{aligned} mbox(d, e) &= H_M(\texttt{rings.mailbox} \| d \| e), \\ entry_M &= (\eta, exp, proof_s, dst, cipher). \end{aligned}$$

Definition 22 (Hidden service descriptor).

$$\begin{aligned} h &= (\nu, \rho, Intro, Relay, \\ &\quad Cap, K, exp, rank), \\ VID(h) &= H_H(h), \\ \mathsf{Lookup}(h) &= P_H^N(h), \\ \mathsf{Valid}(h, \sigma) &= \mathrm{Auth}_H(h, \sigma). \end{aligned}$$

Definition 23 (Subring).

$$\mathcal{S}_u = (u, N_u, \mathrm{owner}_{N_u}, \mathrm{Succ}_{N_u}^{r_u}, E_u)$$

$$\begin{aligned} u &= H_U(policy), \\ memberlog_u &\in \mathsf{MergeLog} \cup \mathsf{FinalState}. \end{aligned}$$

VIII. Cryptographic Carriers and Traffic

Definition 24 (End-to-end carrier).

$$\begin{aligned} |\mathbb{G}| &= q, \\ \mathsf{Scalar}(\mathbb{G}) &= \mathbb{Z}_q, \\ x &\in \mathbb{Z}_q, \\ h &= g^x, \\ \mathbb{G} &\neq R. \end{aligned}$$

Definition 25 (Signed encrypted frame).

$$f = (sid, seq, last, cipher, pk_s, \sigma_s).$$

Algorithm 5 ElGamal Encryption and Decryption

```
Input: Message m, private key x ∈ Z_q, public key h = g^x, generator g
Output: Encrypted message (c_1, c_2)
procedure Encryption
    Choose a random integer k ∈ Z_q
    Compute c_1 = g^k
    Compute c_2 = m · h^k
    return (c_1, c_2)
procedure Decryption
    Compute m = c_2 · (c_1)^{-x}
    return m
```

$$m_f = sid \| seq \| last \| cipher,$$
$$\text{FrameValid}(f) \Leftrightarrow V(pk_s, \sigma_s, m_f) = \top \wedge \text{Session}(D(pk_s), sid, e).$$

Definition 26 (Secret sharing placement).

$$\begin{aligned} P &\in \mathbb{F}_q[X], \\ sh_i &= (i, P(i)) \in \mathbb{F}_q^2, \\ H_S(sh_i) &\in R, \\ \text{reconstruct}(\{sh_i\}) &= P(0) \quad (\mathbb{F}_q\text{-interp.}) \end{aligned}$$

This is Shamir reconstruction over the separate field carrier [7].

Definition 27 (Relay frame).

$$r = (path, dst, \nu, session, payload, report).$$
$$\begin{aligned} \text{RelayValid}(r) &= \text{SessionValid}(session) \wedge dst \in R, \\ \text{SplitVocabulary} &= \mathsf{MSRP}. \end{aligned}$$

The split vocabulary follows MSRP [8].

Algorithm 6 End-to-End Chunking Algorithm

```
procedure E2E Chunking
    Input: data, chunk size
    Output: chunks
    chunks ← []
    total-chunks ← ceil(len(data)/chunk-size)
    for i in range(total-chunks) do
        chunk ← data[i*chunk-size : (i+1)*chunk-size]
        append chunk to chunks
    return chunks
```

Invariant 5 (Chunk reassembly).

$$\text{complete}(m) \Leftrightarrow \forall i < total(m).\ \exists! c_i \wedge \sum_i |c_i| \le B_m \wedge now < ttl(m).$$

IX. Ordering, Ranking, and Security

Definition 28 (Sidecar witness).

$$w = (domain, root, height, time, finality).$$
$$c = H(msg),$$
$$\text{Witness}(msg, w) \Leftrightarrow \text{Include}(c, w.root) = \top \wedge \text{Final}(w) = \top.$$

Definition 29 (Witness order).

$$a \prec b \Leftrightarrow domain(w_a) = domain(w_b) \wedge time(w_a) < time(w_b).$$
$$\chi : Domain_a \times Domain_b \to Order.$$

Definition 30 (Ranking event).

$$q = (subject, rater, behavior, evidence, epoch, weight, \sigma).$$
$$m_q = subject \| behavior \| evidence \| epoch,$$
$$\text{RankValid}(q) \Leftrightarrow V(k_{rater}, \sigma, m_q) = \top.$$

Definition 31 (Score aggregation).

$$score_e(s) = \text{robust}\{weight(q) \cdot value(q) : q.subject = s\}.$$
$$\text{robust} \in \{\mathsf{median}, \mathsf{trimmedMean}, \mathsf{declared}\}.$$

Attack	Predicate	Rings control
Sybil	many DIDs, one actor	admission cost, ranking, challenge
Replay	stale valid σ	nonce, TTL, session epoch
MITM	wrong key for DID	DID proof, signed envelope, E2E
DoS	unbounded resource use	quotas, ranking, chunk bounds

Table II
Security predicates and controls

Obligation 2 (Replay rejection).

$$\begin{aligned} \text{accept}(s, \eta, t) &\Rightarrow \eta \notin Seen_s \wedge t < exp(s), \\ Seen'_s &= Seen_s \cup \{\eta\}, \\ \mathsf{emitDelivery} &< Seen'_s. \end{aligned}$$

X. Verification and Evaluation

Definition 32 (State relation).

$$C = (S, Q)$$
$$\frac{Q = a :: Q_0 \quad I_A(a) = i \quad \tau(S, i) = (S', O, \beta)}{(S, Q) \to (S', Q_0 \cdot \beta)}$$
$$\mathsf{Explore} = \mathsf{TLAStyle}(\to)$$

The exploration relation follows TLA+ style and Stateright execution [9], [10].

Invariant 6 (Effect refinement).

$$\text{Impl}_A(s, i) \preceq I_A^\sharp(\tau(s, i)).$$
$$\pi_{state}(\text{Impl}_A) = \pi_{state}(I_A^\sharp \circ \tau).$$

Obligation 3 (Topology coverage).

$$\mathcal{L} = \{line, wrap, gap, partition, merge, churn\}$$

$$\mathcal{S} = \{loss, dup, reorder, timeout, restart\}.$$

$$\mathsf{CheckCase} = (L, S, P), \qquad L \in \mathcal{L},\ S \in \mathcal{S}.$$

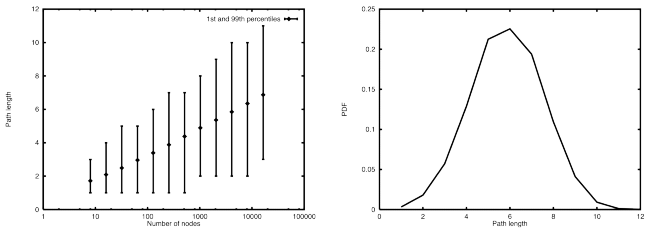
Definition 33 (Latency decomposition).

$$T_{total} = T_{conn} + h \cdot T_{hop} + T_{crypto} + T_{app} + T_{witness}.$$

$$h = O(\log |N|) \quad (F_n \text{ populated}).$$

Fraction of failed nodes	Mean routing hops	Mean num. of lookup timeouts
0	3.84	0.0
0.1	4.03	0.60
0.2	4.22	1.17

Table III
Latency on failed node rate of Chord [4]



(a) (b)

Figure 8. Chord lookup path [4]

Definition 34 (Reliability baseline).

$$\begin{aligned} failrate &= \frac{|F|}{|Q|}, \\ timeoutMean &= \frac{1}{|Q|} \sum_{q \in Q} to(q). \end{aligned}$$

join/leave rate (per sec./stab. period)	Mean routing hops	Mean num. of lookup timeouts	Lookup failures per 10k
0.05/1.5	3.90	0.5	0
0.10/2	3.83	0.11	0
0.15/4.5	3.84	0.16	2
0.20/6	3.81	0.23	5
0.25/7.5	3.83	0.30	6
0.30/9	3.91	0.34	8
0.35/10.5	3.94	0.42	16
0.40/12	4.06	0.46	15

Table IV
Failure rate of Chord [4]

Obligation 4 (Evaluation morphism).

$$\begin{aligned} \Phi_{eval} :& (h, timeouts, failures)_{Chord} \\ & \mapsto (T_{total}, timeoutMean, failrate)_{Rings}, \end{aligned}$$

$$\Delta\Phi = T_{conn} + T_{crypto} + T_{witness} + T_{app}.$$

XI. Related Work

Distributed hash tables give Rings its closest technical ancestry. IPFS and libp2p use Kademlia-derived routing, where distance is measured by XOR and routing state is organized into buckets [11]–[13]. This is a strong fit for content-addressed lookup, but Rings needs a different law: the owner of a DID or VID is the first live node clockwise from that point, and replica ownership is obtained by successor-list extension. The paper therefore uses Chord's circular order as the base model, then uses Zave's CorrectChord results to avoid treating the original Chord maintenance protocol as already correct under churn [4], [5].

Relay anonymity systems take a different route. Tor and Nym route traffic over selected relay paths and focus on hiding communication metadata from local observers or intermediate services [14], [15]. Rings does not replace these systems as anonymity systems. It instead specifies a structured ownership substrate on which relay, mailbox, and hidden-service style protocols can be written. This distinction matters: a relay path can carry traffic, but it does not define the algebraic owner of a DID, mailbox, chunk, or service descriptor.

Blockchain and account-based decentralized systems provide another partial answer. A ledger can publish names, order events, and make writes globally auditable, but the price is global consensus on operations that often only need local routing, replicated storage, or protocol-specific ordering. Rings keeps global consensus outside the base overlay. When a protocol needs stronger ordering evidence, it uses sidecar witnesses rather than changing the overlay into a blockchain.

Browser-native networking is also part of the design space. WebRTC gives browsers a standardized peer transport, while WebAssembly gives portable code a browser execution target [16], [17]. Rings treats these as adapters for the same abstract protocol object, not as separate networks. The same DID/session, Chord owner, effect, and resource-placement laws should hold for browser nodes and native nodes; only the IO interpreter changes.

System	Metric	Ownership law
Kademlia fam.	XOR	bucket routing
Relay mixnets	path	no structured owner
Rings	circular	owner, Succ, Rep

Table V
Routing and ownership comparison [11]–[15]

XII. Conclusion

Rings is an algebraic structured peer-to-peer network: structured because ownership, routing, replication, and repair are defined over CorrectChord's circular overlay, and algebraic because each layer preserves an explicit carrier or morphism rather than hiding semantics inside ad hoc network procedures.

$$\mathcal{R}\rangle\backslash\}\int = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X}).$$

$$\begin{aligned}
\mathsf{Overlay} &= \mathsf{CorrectChord}(R, N, r),\\
\mathsf{Resource} &= (X_\nu, H_\nu) \in \mathbf{Set}/R,\\
\mathsf{Placement} &= \Omega_N(X_\nu, H_\nu),\\
\mathsf{Transition} &= \tau_\nu : S_\nu \times I_\nu \to Y_\nu,\\
Y_\nu &= \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu),\\
\mathsf{Crypto} &= \mathbb{G} \vee \mathbb{F}_q, \quad \mathsf{Crypto} \cap R = \varnothing,\\
\mathsf{Evidence} &= \{\mathsf{figures}, \mathsf{tables}, \mathsf{algorithms}\}\\
&\quad \cup \{\mathsf{invariants}, \mathsf{obligations}\}.
\end{aligned}$$

References

[1] J. Kshetri, "Data privacy and security risks of cloud computing for consumers," International Journal of Information Management, vol. 34, no. 6, pp. 607–615, Dec 2014.

[2] E. Kokkonen and A. Porras, "Data privacy and security in cloud computing: a review of the state-of-the-art," Computer Science Review, vol. 28, pp. 21–38, Oct 2018.

[3] M. R. Grimaila, "A comparison of data privacy regulations: the eu and the us," Journal of International Commerce, Economics and Policy, vol. 7, no. 3, pp. 1–27, 2016.

[4] I. Stoica, R. Morris, and D. Karger, "Chord: A scalable peer-to-peer lookup service for internet applications," https://dl.acm.org/doi/10.1145/383059.383071, 2001.

[5] P. Zave, "Reasoning about identifier spaces: How to make chord correct," IEEE Transactions on Software Engineering, vol. 43, no. 12, pp. 1144–1156, 2017, https://arxiv.org/abs/1610.01140.

[6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Stabilization, Safety, and Security of Distributed Systems, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[7] A. Shamir, "How to share a secret," https://link.springer.com/article/10.1007/BF00185039, 1979.

[8] B. Campbell, R. Mahy, and C. Jennings, "The message session relay protocol (msrp)," RFC 4975, 2007, https://www.rfc-editor.org/info/rfc4975.

[9] L. Lamport, Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.

[10] J. Nadal, "Stateright: Model checking for implementing distributed systems," 2026, https://www.stateright.rs/.

[11] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the xor metric," International Workshop on Peer-to-Peer Systems, pp. 53–65, 2002.

[12] J. Benet, "Interplanetary file system: A p2p file system for the next web," Proceedings of the 24th International Conference on World Wide Web, pp. 1149–1160, 2015.

[13] J. R. Etheridge, M. Castro, and I. Stoica, "Libp2p: A modular network stack for p2p systems," USENIX Association Conference on Networked Systems Design and Implementation, vol. 14, pp. 1–15, 2017.

[14] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," ACM Transactions on Computer Systems, vol. 22, no. 3, pp. 303–327, 2004.

[15] N. D. Team, "Nym: A decentralized mix network," Journal of Privacy and Security, vol. 6, no. 2, pp. 135–144, 2021.

[16] Internet Engineering Task Force, "WebRTC: Real-Time Communications between Browsers," 2021, work in Progress. [Online]. Available: https://tools.ietf.org/html/draft-ietf-rtcweb-overview-26

[17] W. C. Group, "WebAssembly," 2017. [Online]. Available: https://webassembly.org/